



Непрерывная разработка и интеграция CI/CD

...

Лекция 7

Волков Михаил Юрьевич

Тема 7

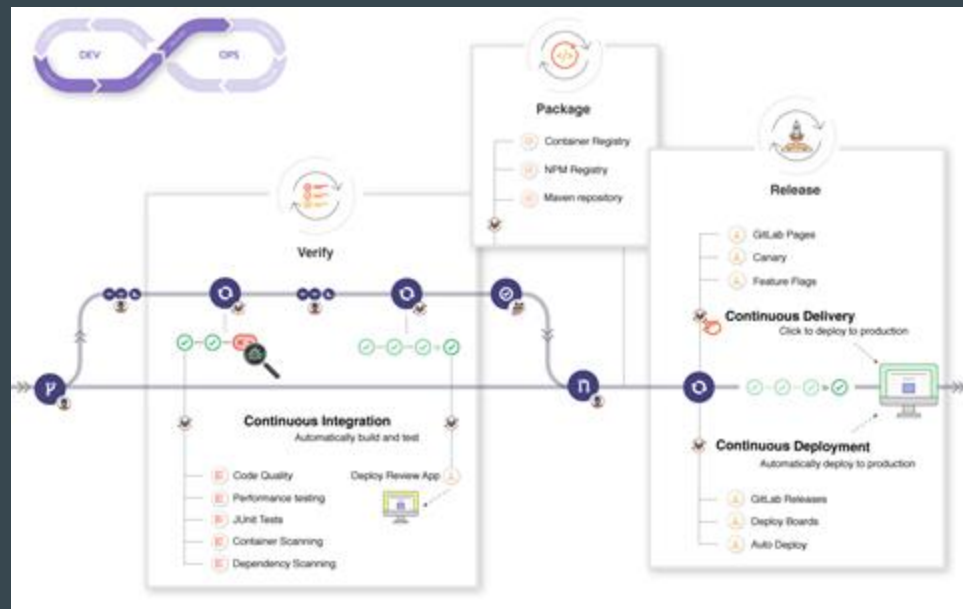
Приемы работы с Gitlab CI

Ключевые особенности GitLab

- Совместимость;
- Метки и документация;
- Гибкие настройки доступа;
- Удобный импорт и экспорт данных;
- Kubernetes по умолчанию для развертывания;
- Выделенное пространство в облаке;
- Инструменты аналитики и планирования;
- Регулярные обновления.

GitLab CI/CD

CI/CD — технология непрерывной интеграции и доставки. CI/CD помогает автоматизировать и масштабировать проекты, что значительно сокращает время разработки. GitLab CI/CD — инструмент, который позволяет использовать GitLab в полноценную платформу для DevOps со всем необходимым функционалом.



Сравнение GitLab CI/CD и GitHub Actions

Основные различия между этими инструментами:

- В CI/CD скрипты в заданиях выполняются с помощью ключа `script`, а в Actions для этого используется ключ `run`.
- Задания на разных платформах в CI/CD выполняются с помощью ключа `tags`, а в Actions — с помощью `runs-on`.
- Оба инструмента могут работать с заданиями в образах Docker, а также указывать дополнительные контейнеры, для чего в CI/CD используется ключ `image`, а Actions — `container`.
- При выполнении заданий с условиями CI/CD использует ключ `rules`, а Actions — `if`.
- Для выполнения заданий с зависимостями в CI/CD есть ключ `stages`, а в Actions используется `needs`.

Шаблоны

Минимальный сценарий. Для небольших заданий, состоящих из пары заданий:

```
stages:
```

```
- build
```

```
TASK_NAME:
```

```
stage: build
```

```
script:
```

```
- ./build_script.sh
```

- stages — описание стадий нашего пайплайна. В данном примере, всего одна.
- TASK_NAME — имя нашей задачи.
 - stage — стадия, к которой относится наше задание.
 - script — набор скриптов для выполнения.

Шаблоны

Стандартный цикл при сборке. Обычно, процесс CI/CD включает следующие шаги:

- Сборка пакета.
- Тестирование.
- Доставка.
- Развертывание.

```
stages:  
  - build  
  - test  
  - delivery  
  - deploy
```

```
build-job:  
  stage: build  
  script:  
    - echo "Start build job"  
    - build-script.sh
```

```
test-job:  
  stage: test  
  script:  
    - echo "Start test job"  
    - test-script.sh
```

```
delivery-job:  
  stage: delivery  
  script:  
    - echo "Start delivery job"  
    - delivery-script.sh
```

```
deploy-job:  
  stage: deploy  
  script:  
    - echo "Start deploy job"  
    - deploy-script.sh
```

Задания

Общий синтаксис опций, которые могут применяться при описании задания:

```
<TASK_NAME>:
```

```
<OPTION1>: ...
```

```
<OPTION2>: ...
```


Задания

Опция Stage указывает, к какой стадии относится задание:

```
stages:
```

```
- build
```

```
- test
```

```
TASK_NAME:
```

```
...
```

```
stage: build
```

```
TASK_NAME:
```

```
...
```

```
stage: test
```

Задания

Стадии описываются в общей директиве stages.

Также есть две стадии, которые не требуют предварительного определения в stages:

- .pre — запускает до выполнения основных заданий конвейера.
- .post — выполняется в конце, после выполнения основных заданий нашего пайплайна.

```
stages:
```

```
- build
```

```
- test
```

```
getVersion:
```

```
  stage: .pre
```

```
  script:
```

```
    - VERSION=$(cat  
VERSION_FILE")
```

```
    - echo "VERSION=${VERSION}" >  
variables.env
```

```
  artifacts:
```

```
  reports:
```

```
  dotenv: variables.env
```

Задания

Image задает имя образа, если наше задание выполняется в контейнере docker:

```
TASK_NAME:
```

```
...
```

```
image: debian:11
```

Задания

Before_script опция определяет список команд, которые должны выполняться перед опцией script и после получения артефактов.

```
TASK_NAME:
```

```
...
```

```
before_script:
```

```
- echo "Run before_script"
```

Задания

Script – основная часть, где выполняются задания сценария, описана в опции script.

1. Описание массива команд. Мы просто перечисляем списком те команды, которые необходимо последовательно выполнить нашему сценарию в конкретной задаче:

2. Длинные команды, разбитые на несколько строк. Нам может потребоваться выполнить команды в виде скрипта (с операторами сравнения, например). В этом случае будет удобна многострочная запись. Ее можно указать разными способами:

1. Индикатор | :
2. Индикатор > :

```
1. TASK_NAME:
   ...
   script:
   - command1
   - command2
```

```
2.1. TASK_NAME:
     ...
     script:
     - |
       command_line1
       command_line2
```

```
2.2. TASK_NAME:
     ...
     script:
     - >
       command_line1
       command_line1 continue

       command_line2
       command_line2 continue
```

Задания

After_script – набор команд, которые запускаются после scripts, даже, при неудачном завершении последнего.

```
TASK_NAME:
```

```
...
```

```
script:
```

```
...
```

```
after_script:
```

```
- command1
```

```
- command2
```

Задания

Artifacts представляют из себя промежуточные сборки или файлы, которые могут передаваться от одного этапа – другому.

1. Можно указать, какие файлы или каталоги будут артефактами:

```
TASK_NAME:
...
artifacts:
  paths:
    - ${PKG_NAME}.deb
    - ${PKG_NAME}.rpm
    - *.txt
    - configs/
```

```
TASK_NAME 2:
...
script:
  - cat *.txt
  - yum -y localinstall
  ${PKG_NAME}.rpm
  - apt -y install
  ./${PKG_NAME}.deb
```

Задания

Artifacts представляют из себя промежуточные сборки или файлы, которые могут передаваться от одного этапа – другому.

2. Можем передать системные переменные, которые были нами переданы в файл:

```
TASK_NAME:
```

```
...
```

```
script:
```

```
- echo -e "VERSION=1.1.1"
```

```
> variables.env
```

```
...
```

```
artifacts:
```

```
reports:
```

```
dotenv: variables.env
```


Задания

Artifacts представляют из себя промежуточные сборки или файлы, которые могут передаваться от одного этапа – другому.

3. При необходимости, можно исключить из списка определенные файлы по названию или маске:

```
TASK_NAME:
```

```
...
```

```
artifacts:
```

```
paths:
```

```
...
```

```
exclude:
```

```
- ../.git/**/*
```

Задания

Extends позволяет вынести часть сценария в отдельный блок и объединить его с заданием.

```
.my_extend:
```

```
  stage: build
```

```
  variables:
```

```
    USERNAME: my_user
```

```
  script:
```

```
    - extend script
```

```
TASK_NAME:
```

```
  extends: .my_extend
```

```
  variables:
```

```
    VERSION: 123
```

```
    PASSWORD: my_pwd
```

```
  script:
```

```
    - task script
```

Задания

В задании TASK_NAME мы используем extends. В результате, задание примет такой вид:

```
TASK_NAME:
  stage: build
  variables:
    VERSION: 123
    PASSWORD: my_pwd
    USERNAME: my_user
  script:
    - task script
```

Изменения:

- stage: build попало в задание из my_extend.
- произошло объединение variables, итог в задание попали VERSION, PASSWORD и USERNAME.
- script используется из задания (значения ключей не объединяются, приоритет за значением основного задания).

Задания

Environment позволяет указать системную переменную, которая будет создана для выполнения задания.

```
TASK_NAME:
```

```
...
```

```
environment:
```

```
RSYNC_PASSWORD: rpass
```

```
EDITOR: vi
```

Задания

Release публикует релиз на портале Gitlab
для нашего проекта.

```
TASK_NAME:
```

```
...
```

```
release:
```

```
  name: 'Release $CI_COMMIT_TAG'
```

```
  tag_name: '$CI_COMMIT_TAG'
```

```
  description: 'Created using the  
release-cli'
```

```
  assets:
```

```
    links:
```

```
      - name: "myprogram-${VERSION}"
```

```
        url:
```

```
"https://gitlab.com/master.dmosk/proje  
ct/-  
/jobs/${CI_JOB_ID}/artifacts/raw/mypro  
gram-${VERSION}.tar.gz"
```

```
  rules:
```

```
    - if: $CI_COMMIT_TAG
```

Спасибо за внимание !

